

test runners like Gauge, Jest or Mocha.

Taiko controls Chromium browsers by sending commands over the Chrome DevTools protocol. The DevTools protocol is a low-level API to interact with Chromium via WebSockets. For example, here's a script for reading the URL of the page currently displayed on Chromium.

```
const devtools = new WebSocket(
  'ws://localhost:9222/devtools/
  page/69990451-aaab-4ef8-87b1-
  ea77b8101b2a'
);
devtools.onopen = function() {console.
log('websocket is connected ...');
ws.send('{"id":1,"method":"Page.
enable","params":{}}', err => {
  if (err) {
    console.log(err);
  } else {
    ws.send(
      '{"id":2,"method":"Page.navigate
", "params":{"url":"https://taiko.gauge.
org/"}}'
    );
  }
});
};

devtools.onmessage = ({ data }) => {
  const {
    result: {
      result: { value }
    }
  } = JSON.parse(data);
  console.log('WebSocket Message
Received: ', value);
  if (response.id == 3) {
    const buffer = new Buffer(response.
result.data, 'base64');
    fs.writeFile('./screenshots/ws_
screenshot.png', buffer, function(err)
{
      if (err) {
        return console.log(err);
      }
      console.log('The ScreenShot is
Captured');
      ws.close();
    });
  });
};
```

```
npm install -g taiko

/Users/****/.node/bin/taiko → /Users/****/.node/lib/node_modules/taiko/bin/taiko.js

> taiko@1.0.2 install /Users/****/.node/lib/node_modules/taiko
> node lib/install.js

Downloading Chromium r662092 - 88.3 Mb [=====] 100% 0.0s
Chromium downloaded to /Users/****/.node/lib/node_modules/taiko/.local-chromium/mac-662092
+ taiko@1.0.2
updated 1 package in 8.233s
```

Figure 2: Installing Taiko

```
}
};

devtools.send(
  JSON.stringify({
    id: 1,
    method: 'Runtime.evaluate',
    params: {
      expression: `The current URL is:
' + location.href`
    }
  })
);

devtools.send('{"id":3,"method":"Page.
captureScreenshot","params":{}}', err
=> {
  if (err) {
    console.log(err);
  }
});

Even though the plain DevTools
protocol APIs are powerful, they are
also verbose, and hard to write and
maintain. Taiko provides a simpler
user interaction based API to automate
the browser. Let's look at how Taiko
simplifies the above code snippet.

const { openBrowser, goto,
screenshot, currentUrl, closeBrowser } =
require('taiko');
(async () => {
  try {
    await openBrowser();
    await goto('https://taiko.gauge.
org/');
    await currentUrl();
    await screenshot();
  } catch (error) {
    console.error(error);
  } finally {
    await closeBrowser();
  }
})();

To see how Taiko fits in your test
setup, let's look at its high-level layers
first (please refer to Figure 3).
```

Using Taiko's superpowers to create reliable tests

Interactive REPL (read-eval-print loop) creates clean code. Taiko has an interactive REPL to record tests and generate clean JavaScript code.

To launch the REPL, type *taiko* in your favourite command line. The REPL is now ready to accept Taiko's API as commands for controlling the browser. To see the list of commands, just type `api`.

Explicit waits become obsolete

Taiko's API understands actions that trigger HTTP or HTTPS requests to a Web server, and loads the server response data back into the script or fetches dynamic content. This results in an implicit understanding of when a page is ready, and the user doesn't have to add explicit waits in between each command and page navigation.

Proximity selectors keep your tests independent of the underlying code. Taiko's smart selectors help test Web applications without depending on the internal app structure. Taiko works in a complete 'black box' way.

```
await click(link("Block",
toLeftOf("name"))
```